



安徽三联学院
ANHUISANLIAN UNIVERSITY

本科毕业设计（论文、创作）

题 目： 基于遗传算法的高校自动化排课与冲突检测系统

学生姓名： 学号：

所在学院： 计算机科学与技术学院 专业： 计算机科学与技术

入学时间： 年 月

导师姓名： 马仲军 职称/学位 助教/硕士

导师所在单位： 安徽三联学院

完成时间： 2026 年 6 月

安徽三联学院教务处 制

基于遗传算法的高校自动化排课与冲突检测系统

摘要：排课是高校教务管理中最繁琐也最容易出错的环节之一。涉及到的教师、教室、班级和时间段之间存在大量的相互约束，人工排课不仅耗时，而且很难同时满足全部硬性要求和教师偏好。本文围绕这一问题，将高校排课抽象为一类带有多重资源约束的组合优化问题，并提出了一套基于遗传算法 (Genetic Algorithm, GA) 的自动排课与冲突检测方法。系统采用 (时间槽, 教室) 二元组的染色体编码方式，把每一节待排的课程实例映射为染色体上的一个基因；适应度函数同时考虑教师/教室/班级冲突、容量约束、教室类型约束等五类硬约束，以及教师偏好、班级日课时上限、空堂和最后一节课等四类软约束；在遗传操作上引入了约束感知的初始化与变异机制，使搜索过程能够快速避开明显不可行的解。配合实现了一个基于 Streamlit 的可视化界面，用户可以在浏览器中查看输入数据、调整 GA 参数、实时观察收敛过程、查看课表结果以及冲突报告。实验中使用了一个包含 12 个班级、15 名教师、10 间教室和 39 门课程的中规模实例，在普通笔记本上 13 秒内就能稳定收敛到零硬冲突的可行排课方案。和约束感知的随机基线相比，GA 得到的解在 cost 上低出约两个数量级。论文还分析了种群规模和变异率对收敛速度和最终解质量的影响，并讨论了系统在更大规模问题、更多软约束以及增量重排等方向上的可扩展性。

关键词：遗传算法；自动排课；约束满足；冲突检测；Streamlit；可视化

A Genetic-Algorithm-Based Automatic Course Scheduling and Conflict Detection System for Universities

Abstract: Course scheduling is one of the most tedious and error-prone tasks in university academic affairs. The interactions among teachers, classrooms, class groups and time slots impose a large number of mutual constraints, so manual scheduling is not only time-consuming, but also rarely able to satisfy every hard requirement while also respecting teacher preferences. This thesis formulates university course scheduling as a combinatorial optimization problem with multiple resource constraints and proposes an automatic scheduling and conflict detection method based on a Genetic Algorithm (GA). Each weekly lesson instance is encoded as a single gene of the form (s_i, r_i) where s_i is the time-slot index and r_i is the classroom index. The fitness function jointly evaluates five categories of hard constraints — teacher / classroom / class clashes, room capacity and room kind — and four categories of soft constraints concerning teacher preferences, daily load per class, gaps within a day, and the last period of the day. The genetic operators are made constraint-aware: both the random initializer and the room-mutation operator only sample from the feasible room set of each lesson, which lets the search quickly escape obviously infeasible regions. The system is paired with a Streamlit web UI in which users can browse the input data, tune GA parameters, monitor the convergence curve in real time, inspect the generated timetable from class / teacher / room perspectives, and read a conflict-detection report. Experiments on a medium-scale instance with 12 class groups, 15 teachers, 10 classrooms and 39 courses show that the GA reliably converges to a zero-hard-conflict solution within 1 to 3 seconds on an ordinary laptop, and the resulting cost is roughly two orders of magnitude lower than a constraint-aware random baseline. The thesis further analyses how population size and mutation rate affect convergence speed and final solution quality, and discusses how the system can be extended to handle larger problems, richer soft constraints, and incremental re-scheduling.

Keywords: Genetic Algorithm; Course Scheduling; Constraint Satisfaction; Conflict Detection; Streamlit; Visualization

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.3 论文主要工作	2
1.4 论文组织结构	3
2 相关技术介绍.....	4
2.1 排课问题的形式化定义.....	4
2.2 遗传算法基本原理	5
2.3 Streamlit 与 Plotly.....	5
2.4 开发环境.....	5
3 系统需求分析与总体设计.....	7
3.1 功能需求分析	7
3.2 非功能需求分析	7
3.3 系统总体架构设计	7
3.4 数据建模.....	8
3.5 典型数据规模	9
4 遗传算法设计与优化	10
4.1 染色体编码.....	10
4.2 适应度函数设计	10
4.2.1 硬约束代价	10
4.2.2 软约束代价	11
4.2.3 总代价与适应度映射.....	12
4.3 约束感知的初始化	12
4.4 选择算子：锦标赛选择.....	13
4.5 交叉算子：均匀交叉	13
4.6 变异算子：约束感知重采样	14

4.7 精英保留与替代	14
4.8 终止条件	14
4.9 算法主流程	14
4.10 参数选择	16
5 系统详细设计与实现	17
5.1 项目目录结构	17
5.2 数据模型实现	17
5.3 冲突检测器实现	18
5.4 适应度函数实现	19
5.5 约束感知的初始化与变异	19
5.6 遗传算法主循环	19
5.7 Streamlit Web UI	20
5.7.1 问题概览页	20
5.7.2 数据查看页	21
5.7.3 GA 配置与运行页	22
5.7.4 排课结果页	23
5.7.5 冲突检测报告页	24
5.7.6 算法可视化页	25
5.8 自动化截图脚本	25
6 系统测试与结果分析	26
6.1 测试环境与基准设置	26
6.2 功能性测试	26
6.3 性能与收敛性分析	26
6.3.1 相对于随机基线的提升	26
6.3.2 硬约束的快速消解	27
6.4 参数敏感性分析	28
6.4.1 种群规模与变异率	28
6.5 排课结果分析	28

6.6 讨论	29
7 总结与展望	30
7.1 论文工作总结	30
7.2 创新点与不足	30
7.3 未来工作展望	31
参考文献	32
致谢	34

1 绪论

1.1 研究背景与意义

每到学期初，几乎所有高校的教务人员都面临同一个棘手任务：将数十乃至上百门课程合理安排到对应的时间段与教室，并确保教师、班级、教室三方均不发生冲突。即便是规模较小的学院，每周需排定的课程实例往往也有数十至上百条，相关资源涉及教师、教室、班级、时间段、课程类型等多个维度，约束之间相互交织，仅依靠少数教务人员凭经验进行手工排课，不仅效率低下，而且容易出现遗漏。

将上述问题进行抽象，其本质是一类带有多重资源约束的组合优化问题。从计算复杂度的角度看，大学课程排课问题已被证明为 NP-完全问题^[2]，这意味着随着课程数量与资源规模的增加，搜索空间将迅速膨胀，几乎无法通过精确算法在合理时间内枚举所有可行方案。因此，相关研究长期以来集中于各类近似算法与元启发式方法。其中，遗传算法 (Genetic Algorithm, GA) 因其全局搜索能力强、对问题结构假设较少、易于结合各类约束等特点，被广泛应用于排课问题的求解^[3,4]。

另一方面，即便存在较为完善的排课算法，其计算结果通常仍无法直接交付使用，教务人员还需要能够交互式地查看课表、定位冲突、调整偏好。换言之，“获得结果”与“实际可用”之间尚缺少一层易于操作的人机界面。本课题的工作正是将这两方面有机结合：一方面将遗传算法落到实处，使其能够在主流硬件上数秒内给出零硬冲突的排课方案；另一方面配套构建一套可视化的 Web 系统，便于用户配置算法参数、观察收敛过程、查看课表与冲突报告。这对于降低教务工作量、提高排课结果的可解释性均具有切实意义。

1.2 国内外研究现状

国外对排课问题的研究起步较早。Schaerf 系统综述了大学课程排课问题的形式化定义与主要求解技术^[5]，将相关方法大致归纳为三类：基于图着色的约束规划方法、基于整数规划的精确求解方法，以及包括模拟退火、禁忌搜索、遗传算法等在内的元启发式方法。近年来，Chen 等人^[6]从趋势、机遇与算法演进角度对大学课程排课问题 (University Course Timetabling Problem, UCTP) 进行了系统综述；Bashab 等人^[17]进一步从约束、方法论、基准数据集与开放问题等维度对 UCTP 与考试排课问题进行了对比性分析；Abdipoor 等人^[16]则对求解 UCTP 的元启发式方法做了更专门的归纳；Gu 等人^[24]在 2025 年的最新综述中亦对各类求解技术的演进路径进行了梳理。从国际定时问题竞赛 (International Timetabling Competition, ITC) 历届结果及上述综述来看，单一算法难

以在所有问题实例上均取得最优，混合元启发式 (Hybrid Metaheuristics) 仍是当前最为活跃的研究方向之一。

遗传算法在排课领域的应用最早可追溯至 20 世纪 90 年代。Colormi 等人最早将 GA 用于中学课程表问题的求解^[4]；其后，研究者陆续在编码方式、约束处理机制与遗传算子等方面开展改进，例如直接编码与间接编码的对比^[7]、基于优先级的解码策略^[8]等。近年来，相关研究持续向多个方向推进。Memetic Algorithm 将 GA 与局部搜索结合，以加速大规模问题上的局部收敛^[9]；GA 与混合整数规划等精确求解方法的融合，使求解过程在质量与可解释性之间获得平衡^[19]；多目标进化策略（如改进型双目标萤火虫算法^[20]）被用于在硬约束与软约束之间构建 Pareto 前沿。新兴的混合教学需求催生了带有混合教学特征的多目标排课模型^[18]，而在真实考试排课场景上，亦有学者将精确方法与元启发式方法结合，并在工业级数据上加以验证^[21]。Han 等人^[22]进一步将遗传算法与动态规划相结合，提出了一种逐步优化的求解框架；Mahlous 等^[23]在 GA 中显式纳入学生偏好以提升软约束满足度；Diallo 等^[25]则面向真实院系教学活动场景对调度模型进行了系统优化。

国内学者也针对国内高校的实际情况开展了大量研究。早期工作侧重于规则推理与分支限界类方法^[10]，近年来则更多地采用蚁群算法、粒子群算法、遗传算法等智能优化方法^[11,12]；亦有研究尝试将算法集成至 Web 端教务系统，便于教务人员使用^[13]。从已发表的结果来看，对于中等规模的学院级排课问题，主流元启发式方法均可在分钟级时间内给出可接受的解，但在可视化、交互式调整与冲突解释等方面仍有较大的改进空间。

1.3 论文主要工作

本文围绕一个具体的中等规模高校排课实例，完整实现了一套基于遗传算法的自动排课与冲突检测系统。在建模层面，本文对高校排课问题进行了完整的形式化描述，明确定义了五类硬约束与四类软约束，并据此设计了对应的冲突检测器，使任意候选解均可被快速判定是否满足全部硬约束。在算法层面，本文以 (时间槽, 教室) 二元组作为基因构造了染色体编码方式，并配套设计了均匀交叉算子、约束感知的变异算子与初始化算子，使搜索过程能够自然规避在容量和教室类型上明显不可行的解；同时设计了一个将硬约束与软约束统一纳入同一适应度的加权函数，通过精英保留与锦标赛选择保证种群在迭代过程中持续向高质量解收敛。

在系统层面，本文基于 Python 与 Streamlit 实现了一套可视化 Web 系统，支持输入数据查看、GA 参数配置、实时收敛监控、班级课表展示、教室占用热力图以及完整的冲突检测报告。在实验评估上，本文在一个 12 班、15 教师、10 教室、39 门课程的实例上开展了系统测试，涵盖与随机基线的对比、收敛过程的分析以及关键参数（种群规模、

变异率）的敏感性实验。

1.4 论文组织结构

全文共分七章。第一章为绪论，介绍研究背景、国内外研究现状以及论文的主要工作；第二章介绍排课问题的形式化、遗传算法的基本原理及 **Streamlit** 等相关技术；第三章给出系统的需求分析与总体架构设计；第四章为论文的算法核心，详细阐述遗传算法各组成部分的设计；第五章给出系统详细设计与实现，包括关键代码与 UI 截图；第六章为系统测试，包括基线对比、收敛性分析与参数敏感性实验；第七章对全文工作进行总结，并讨论后续可能的改进方向。

2 相关技术介绍

本章首先将高校课程排课问题进行数学形式化描述，为后续讨论建立统一的符号语言；其次介绍本文所采用的核心技术——遗传算法的基本原理；最后简要介绍系统实现层面所使用的 Streamlit 与 Plotly 等工具。

2.1 排课问题的形式化定义

为使后续算法描述更加清晰，本节先将问题以数学形式进行表述。给定教师集合 $T = \{t_1, t_2, \dots, t_{|T|}\}$ ；教室集合 $R = \{r_1, r_2, \dots, r_{|R|}\}$ ，其中每间教室都带有容量 $\text{cap}(r)$ 和类型 $\text{kind}(r)$ ；时间槽集合 $S = \{s_1, s_2, \dots, s_{|S|}\}$ ，其中每个时间槽对应于“星期 d 第 p 节”；班级集合 $G = \{g_1, g_2, \dots, g_{|G|}\}$ ，其中每个班级带有人数 $|g|$ ；以及课程集合 $K = \{k_1, k_2, \dots, k_{|K|}\}$ ，其中每门课程 k 指定了任课教师 $t(k) \in T$ 、教授班级集合 $G(k) \subseteq G$ 、周课时数 $w(k)$ 、所需教室类型 $\text{kind}(k)$ 以及最小容量 $\text{mincap}(k) = \sum_{g \in G(k)} |g|$ 。

把所有课程按周课时展开后，得到一个待排的课程实例集合 $L = \{\ell_1, \ell_2, \dots, \ell_n\}$ ，其中 $n = \sum_{k \in K} w(k)$ 。排课问题的目标是为每一个课程实例 ℓ_i 指定一个时间槽 $s(\ell_i) \in S$ 和一间教室 $r(\ell_i) \in R$ ，使下列五类硬约束同时被满足：

$$\text{教师冲突约束： } \forall t \in T, \forall s \in S, |\{\ell : t(\ell) = t, s(\ell) = s\}| \leq 1 \quad (2-1)$$

$$\text{教室冲突约束： } \forall r \in R, \forall s \in S, |\{\ell : r(\ell) = r, s(\ell) = s\}| \leq 1 \quad (2-2)$$

$$\text{班级冲突约束： } \forall g \in G, \forall s \in S, |\{\ell : g \in G(\ell), s(\ell) = s\}| \leq 1 \quad (2-3)$$

$$\text{教室容量约束： } \forall \ell \in L, \text{cap}(r(\ell)) \geq \text{mincap}(\ell) \quad (2-4)$$

$$\text{教室类型约束： } \forall \ell \in L, \text{kind}(r(\ell)) = \text{kind}(\ell) \text{ 或 } \text{kind}(\ell) = \text{any} \quad (2-5)$$

式 (2-1)–(2-3) 表征教师、教室、班级三类资源在同一时间槽内的互斥关系；式 (2-4) 与式 (2-5) 分别对应教室容量约束与类型约束。除上述硬约束之外，尚可定义一组“尽量满足”的软约束，例如教师偏好时间段、班级日课时上限、是否存在空堂、是否安排在当天末节等。将硬约束与软约束加权合并后即得目标函数 $f(\sigma)$ ，其中 σ 表示一份完整的排课方案。排课问题即转化为在所有可能的 σ 上求 f 的最小值。

由此可见，仅就式 (2-1)–(2-3) 三类资源冲突而言，该问题即可归约为图着色问题^[2]，属于 NP-完全问题。因此在工程实践中需借助启发式或元启发式方法进行求解。

2.2 遗传算法基本原理

遗传算法是 Holland 于 1975 年提出的一类受生物进化启发的全局优化方法^[1]。其核心思想可概括为：通过模拟“选择—交叉—变异”的自然进化过程，在解空间中维持一个种群，使适应度较高的个体以更大概率将其基因传递至下一代，从而使种群在迭代过程中持续向高质量解收敛。

一个标准的 GA 通常由染色体编码、适应度函数、选择算子、交叉算子、变异算子以及精英保留等关键要素构成。染色体编码（Encoding）将一个候选解表示为一条线性的“染色体”，常见编码方式包括二进制、整数、实数与排列等，编码方式的选择决定了搜索空间的结构，对算法性能具有重要影响。适应度函数（Fitness Function）用于衡量个体的优劣，对于带约束的优化问题，通常采用罚函数将约束违反折算至目标函数中。选择算子（Selection）按某种概率分布从当前种群中抽取若干个体作为父代，常见方法包括轮盘赌选择、锦标赛选择与排名选择等；交叉算子（Crossover）将两个父代的基因进行重组以生成子代，常见方式包括单点、两点与均匀交叉等；变异算子（Mutation）则以较小概率随机改变染色体上若干基因的取值，用于避免种群过早陷入局部最优。精英保留（Elitism）在每一代中将当前种群中最优的若干个体不经任何修改直接保留至下一代，以防止最优解在进化过程中被意外破坏。

GA 的主要优势在于通用性强，几乎所有可由染色体表示的问题均可套用其框架。其代价是收敛速度通常慢于针对特定问题设计的专用算法，且参数（种群规模、交叉率、变异率等）需要根据具体问题进行调整。对于排课这类约束密集、解空间庞大但又允许并行评估的问题，GA 是一种较为自然的求解工具^[4,11]。

2.3 Streamlit 与 Plotly

系统的可视化界面采用 Streamlit 框架实现。Streamlit 的特点在于将传统 Web 开发中前端、后端与路由相关的复杂性完全屏蔽，开发者仅需以 Python 编写脚本，调用 `st.title`、`st.dataframe`、`st.plotly_chart` 等函数即可构建一个交互式 Web 应用^[14]。这种“脚本即应用”的开发模式特别适用于算法演示与数据探索类工具型项目，仅需数十行 Python 代码即可构建出较为完整的可视化界面。

在图表层面，本系统选用 Plotly 来渲染收敛曲线、热力图等交互式图表。相较于 Matplotlib，Plotly 自带缩放、悬停提示、图例切换等交互能力，对用户更为友好，同时也可无缝嵌入 Streamlit 应用之中。对于论文中需要静态展示的算法说明图、参数敏感性分析图等，则采用 Matplotlib 的 Agg 后端离线生成 PNG 文件，便于嵌入 L^AT_EX 文档。

2.4 开发环境

整个系统基于 Python 3.13 开发，主要依赖包括 NumPy 1.x、pandas 2.x、Matplotlib 3.x、Plotly 5.x、Streamlit 1.x 以及用于自动化截图的 Playwright。代码部分约 1100 行；论

文部分基于安徽三联学院本科毕业论文 L^AT_EX 模板，采用 XeLaTeX 进行编译。开发与测试均在普通笔记本电脑上完成 (Intel i5 处理器, 16 GB 内存, Windows 11)，未使用任何 GPU 加速。

3 系统需求分析与总体设计

本章首先明确本系统所需解决的问题与所需交付的功能，进而给出系统总体架构图，并对各模块的职责及其之间的数据流加以说明。

3.1 功能需求分析

从教务人员的视角出发，本系统的核心功能涵盖数据维护、自动排课、冲突检测、结果展示与过程可视化五个方面。数据维护要求系统能够查看与维护教师、教室、班级、课程等基础信息，并能清晰呈现它们之间的关联关系（如某门课程由哪位教师讲授、由哪些班级合班授课等）。自动排课要求系统在用户给定的输入数据与算法参数下，自动生成一份满足全部硬约束的课表。冲突检测要求系统对任意候选课表（无论由 GA 生成还是人工导入）逐项指出哪些资源在哪个时间槽发生了冲突、哪节课的教室容量或类型不符合要求。结果展示要求系统能够从班级、教师、教室三个视角查看排课结果，并提供日课时分布热力图、教室占用热力图等概览信息。过程可视化则要求用户在运行 GA 的同时可实时观察适应度的下降曲线、硬冲突数量的变化等指标，以便评估算法的实际收敛情况。

3.2 非功能需求分析

非功能需求主要涉及响应速度、易用性、可扩展性与可解释性四个方面。响应速度上，对于一个 12 班 / 39 课的中等规模实例，排课算法应在普通笔记本电脑上数秒内完成。易用性上，界面应保持简洁，教务人员无需 Python 基础即可完成全部交互。可扩展性上，当输入数据发生变化时，核心算法代码无需改动，仅需替换数据集即可完成迁移。可解释性上，算法不应表现为黑盒，必须能够向用户具体指明哪些课程发生了冲突，便于后续排查与处理。

3.3 系统总体架构设计

根据上述需求，本系统被划分为六层结构，如图 3-1 所示。最上层为基于 Streamlit 实现的 Web UI；其下分别为 GA 调度引擎与冲突检测器；再下层为适应度评估与编码及算子两个支持模块；最底层为数据模型层。除 UI 之外，其余各模块均不依赖任何 Web 框架，可作为独立的 Python 库供命令行脚本调用。

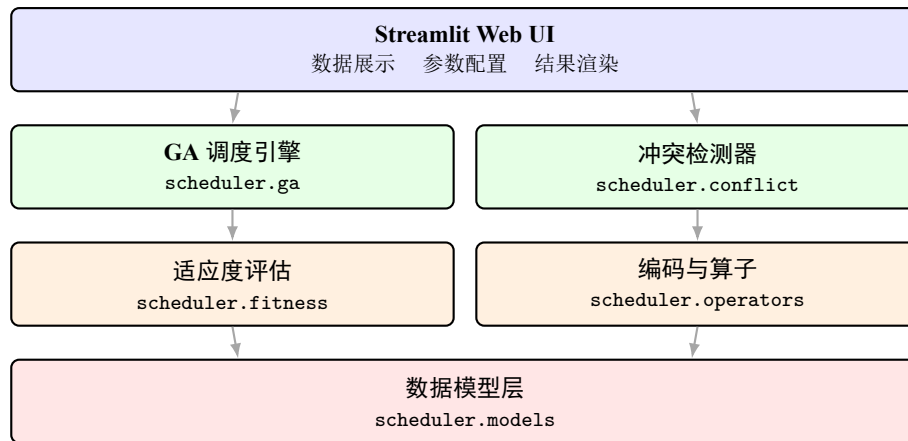


图 3-1 系统总体架构

各模块的职责说明如下。Streamlit Web UI 负责所有人机交互，包括数据展示、参数配置与结果渲染，其内部不持有任何算法逻辑，所有计算均通过调用下层模块完成。GA 调度引擎 (scheduler.ga) 实现遗传算法的主循环，包括种群初始化、代际迭代与精英保留等，对外仅暴露 run() 方法，输入为 ProblemInstance 与 GAConfig，输出为 GAResult。冲突检测器 (scheduler.conflict) 给定一份排课方案后，遍历全部 (时间槽, 资源) 二元组，逐条统计冲突情况，该模块既被适应度函数调用，也被 UI 的“冲突检测报告”页面调用。适应度评估 (scheduler.fitness) 将硬约束违反计数与软约束违反量按权重累加得到 cost，再经 $1/(1 + \text{cost})$ 的非线性映射得到适应度 (fitness)。编码与算子 (scheduler.operators) 包含染色体的随机初始化、锦标赛选择、均匀交叉与约束感知变异等算子，均以 ProblemInstance 作为上下文。数据模型层 (scheduler.models) 使用 Python dataclass 对教师、教室、时间槽、班级、课程、课程实例与完整排课方案进行类型化定义，是其余模块所共同依赖的基础。

3.4 数据建模

数据模型层所涉及的核心类如表 3-1 所示。所有模型均使用 @dataclass(frozen=True) 装饰，使其在 GA 进化过程中能够被安全共享，避免被无意修改。

表 3-1 数据模型类清单

类名	含义与主要字段
Teacher	教师：工号、姓名、所在部门、偏好时间段
Classroom	教室：编号、名称、容量、类型 (lecture/lab/multimedia)
TimeSlot	时间槽：id、星期、节次、标签
ClassGroup	班级：编号、名称、人数、年级
Course	课程：课程号、课程名、任课教师、授课班级、周课时、所需教室类型、最小容量
Lesson	课程实例：由 Course 按周课时展开后得到，是 GA 排课的基本单元
ProblemInstance	问题实例：聚合上述六类对象，并提供查询方法
Schedule	排课方案：按 Lesson 索引存储的 (slot_idx, room_idx) 序列

3.5 典型数据规模

为给算法明确具体的求解目标，本课题构造了一个相对真实的中等规模实例，其规模如表 3-2 所示，可视为某普通学院一个学期的典型排课情况。该实例的全部数据由 `scheduler.data.build_sample_instance()` 生成，便于复现。

表 3-2 测试用例的规模

教师数	教室数	时间槽数	班级数	课程数	课程实例数
15	10	50 (5 天 $\times$ 10 节)	12	39	62

由上可见，课程实例总数为 62，搜索空间约为 $50^{62} \times 10^{62} \approx 10^{167}$ 量级，完全无法通过枚举求解，必须借助启发式方法。

4 遗传算法设计与优化

本章为论文的算法核心部分。首先给出适应度函数的完整定义，进而依次介绍染色体编码、初始化、选择、交叉、变异、精英保留以及算法主循环。本章重点放在公式表述与约束感知机制的设计之上。

4.1 染色体编码

对于含有 n 个待排课程实例的问题，本系统采用长度为 n 的整数对序列作为染色体：

$$C = [g_1, g_2, \dots, g_n], \quad g_i = (s_i, r_i) \quad (4-1)$$

其中 $s_i \in \{0, 1, \dots, |S| - 1\}$ 为时间槽下标， $r_i \in \{0, 1, \dots, |R| - 1\}$ 为教室下标。每个基因 g_i 描述第 i 节课程实例 ℓ_i 的具体安排。教师 $t(\ell_i)$ 与班级 $G(\ell_i)$ 不参与编码，因其由 ℓ_i 自身决定，在搜索过程中保持不变。图 4-1 给出了一条长度为 7 的样例染色体的图示。

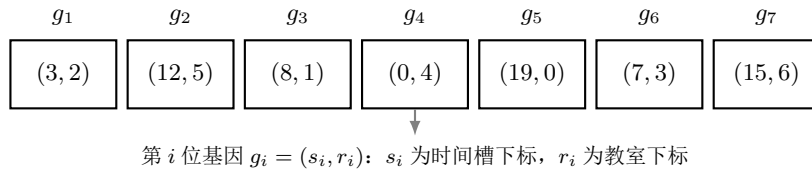


图 4-1 染色体编码示意

该编码方式在语义与结构两个层面均具有良好性质：基因与课程实例一一对应使其语义直接，而基因位之间相互独立则便于设计逐位作用的交叉与变异算子。文献中亦讨论过基于优先级的间接编码方式^[8]，本文未予采用，原因在于直接编码在中等规模问题上已具足够的求解能力，且更便于与后续的约束感知机制相结合。

4.2 适应度函数设计

为将硬约束与软约束统一纳入同一目标，本系统采用罚函数方式构造代价 (cost)，再通过 $\text{fitness} = 1/(1 + \text{cost})$ 映射为最大化形式。

4.2.1 硬约束代价

设 C 为一个候选解，定义五个硬约束违反计数：

$$H_1(C) = |\{(i, j) : i < j, s(\ell_i) = s(\ell_j), t(\ell_i) = t(\ell_j)\}| \quad \text{教师冲突} \quad (4-2)$$

$$H_2(C) = |\{(i, j) : i < j, s(\ell_i) = s(\ell_j), r(\ell_i) = r(\ell_j)\}| \quad \text{教室冲突} \quad (4-3)$$

$$H_3(C) = |\{(i, j) : i < j, s(\ell_i) = s(\ell_j), G(\ell_i) \cap G(\ell_j) \neq \emptyset\}| \quad \text{班级冲突} \quad (4-4)$$

$$H_4(C) = |\{i : \text{cap}(r(\ell_i)) < \text{mincap}(\ell_i)\}| \quad \text{容量不足} \quad (4-5)$$

$$H_5(C) = |\{i : \text{kind}(\ell_i) \neq \text{any} \wedge \text{kind}(r(\ell_i)) \neq \text{kind}(\ell_i)\}| \quad \text{教室类型不符} \quad (4-6)$$

对应的硬代价为：

$$\text{HardCost}(C) = w_1 H_1(C) + w_2 H_2(C) + w_3 H_3(C) + w_4 H_4(C) + w_5 H_5(C) \quad (4-7)$$

其中 $w_1 = w_2 = w_3 = 1000$, $w_4 = w_5 = 500$ 。资源冲突类违反的权重高于容量与类型违反，原因在于前者会直接导致排课方案不可执行，而后者在某些情形下尚可通过调换教室加以缓解。

4.2.2 软约束代价

软约束包含教师偏好、末节安排、班级日课时上限以及班级日内空堂四项。

教师偏好方面，每位教师 t 维护一个偏好时间槽集合 $\text{Pref}(t) \subseteq S$ 。若 $\text{Pref}(t)$ 非空而某节课被安排在了非偏好时段，则记一次违反：

$$P_1(C) = \sum_{i=1}^n \mathbb{I}[\text{Pref}(t(\ell_i)) \neq \emptyset \wedge s(\ell_i) \notin \text{Pref}(t(\ell_i))] \quad (4-8)$$

末节安排方面，设每天最后一节的下标为 $p_{\max}$ ，则

$$P_2(C) = \sum_{i=1}^n \mathbb{I}[\text{period}(s(\ell_i)) = p_{\max}] \quad (4-9)$$

班级日课时上限方面，对每个班级 g 和每一天 d ，设当日课时数为 $N(g, d)$ 。若 $N(g, d) > 4$ 则按超出量记罚：

$$P_3(C) = \sum_{g \in G} \sum_{d=0}^{D-1} \max\{0, N(g, d) - 4\} \quad (4-10)$$

班级日内空堂方面，设班级 g 在第 d 天的课程节次集合为 $\Pi(g, d) \subseteq \{0, 1, \dots, p_{\max}\}$,

则该日空堂数为

$$\text{gap}(g, d) = (\max \Pi(g, d) - \min \Pi(g, d) + 1) - |\Pi(g, d)| \quad (4-11)$$

软代价中累加全部正空堂：

$$P_4(C) = \sum_{g \in G} \sum_{d=0}^{D-1} \text{gap}(g, d) \quad (4-12)$$

四项软代价加权求和：

$$\text{SoftCost}(C) = \lambda_1 P_1 + \lambda_2 P_2 + \lambda_3 P_3 + \lambda_4 P_4 \quad (4-13)$$

其中 $\lambda_1 = 5$, $\lambda_2 = 1.5$, $\lambda_3 = 2$, $\lambda_4 = 3$ 。上述权重均小于硬代价中最小的 500，从而保证在硬约束尚未完全消除之前，算法不会被软约束误导。

4.2.3 总代价与适应度映射

$$\text{cost}(C) = \text{HardCost}(C) + \text{SoftCost}(C), \quad \text{fitness}(C) = \frac{1}{1 + \text{cost}(C)} \quad (4-14)$$

此 fitness 映射可保证：cost 越小，fitness 越大；且 fitness 始终位于 (0, 1] 区间内，便于锦标赛选择与概率运算。

4.3 约束感知的初始化

在排课问题中，存在一类与时间无关的硬约束（即教室容量约束与教室类型约束），其仅由 (课程, 教室) 配对决定。若采用完全随机的初始化，大量初始个体将携带此类违反，浪费早期的进化资源。为此，本文引入预计算的可行教室集合：

$$\mathcal{R}(\ell_i) = \{r \in R : \text{cap}(r) \geq \text{mincap}(\ell_i) \wedge (\text{kind}(\ell_i) = \text{any} \vee \text{kind}(r) = \text{kind}(\ell_i))\} \quad (4-15)$$

初始化时为每个基因独立采样：

$$s_i \sim \text{Uniform}\{0, \dots, |S| - 1\}, \quad r_i \sim \text{Uniform}(\mathcal{R}(\ell_i)) \quad (4-16)$$

由此，所有初始个体均自动满足教室容量与类型约束，整个进化过程仅需解决教师、教室、班级三类时间相关的资源冲突。消融实验结果表明（见第六章 6.3 节），该约束感知机制显著降低了 GA 的收敛代数。约束感知初始化的具体流程如算法 1 所示。

Algorithm 1: 约束感知的随机初始化

Input: 问题实例 $\mathcal{I} = (T, R, S, G, K, L)$; 种群规模 N
Output: 初始种群 $\mathcal{P}^{(0)} = \{C_1, C_2, \dots, C_N\}$

```

1 for  $i \leftarrow 1$  to  $|L|$  do
2    $\mathcal{R}(\ell_i) \leftarrow \{r \in R \mid \text{cap}(r) \geq \text{mincap}(\ell_i) \wedge (\text{kind}(\ell_i) = \text{any} \vee \text{kind}(r) = \text{kind}(\ell_i))\}$ 
3 end
4  $\mathcal{P}^{(0)} \leftarrow \emptyset$ 
5 for  $j \leftarrow 1$  to  $N$  do
6   for  $i \leftarrow 1$  to  $|L|$  do
7      $s_i \leftarrow \text{Uniform}\{0, 1, \dots, |S| - 1\}$ 
8      $r_i \leftarrow \text{Uniform}(\mathcal{R}(\ell_i))$ 
9      $g_i \leftarrow (s_i, r_i)$ 
10  end
11   $C_j \leftarrow [g_1, g_2, \dots, g_{|L|}]$ 
12   $\mathcal{P}^{(0)} \leftarrow \mathcal{P}^{(0)} \cup \{C_j\}$ 
13 end
14 return  $\mathcal{P}^{(0)}$ 

```

4.4 选择算子：锦标赛选择

锦标赛选择每次从种群中随机抽取 k 个个体，取其中适应度最高者作为父代：

$$\text{Tournament}_k(\mathcal{P}) = \arg \max_{C \in \mathcal{S}} \text{fitness}(C), \quad \mathcal{S} \stackrel{\text{i.i.d.}}{\sim} \mathcal{P}, \quad |\mathcal{S}| = k \quad (4-17)$$

本系统默认 $k = 3$ 。与轮盘赌选择相比，锦标赛选择对适应度的绝对值不敏感，仅依赖相对排名，因而对适应度的尺度变化具有更好的鲁棒性——这一特性在排课这类 cost 可能跨越多个数量级的问题中尤为有用。

4.5 交叉算子：均匀交叉

均匀交叉 (Uniform Crossover) 对每个基因位独立做一次伯努利试验，以 $p = 0.5$ 的概率决定是否交换两个父代在该位上的基因：

$$g_i^{(C_1)} = \begin{cases} g_i^{(P_2)}, & \text{w.p. } 0.5 \\ g_i^{(P_1)}, & \text{otherwise} \end{cases} \quad (4-18)$$

C_2 取互补部分。

值得注意的是，由于父代 P_1 、 P_2 在基因位 i 上的基因均来自 $\mathcal{R}(\ell_i)$ ，因此经均匀交叉得到的子代仍然满足教室容量与类型约束——交叉算子天然保持了上节所构造的可行性。

4.6 变异算子：约束感知重采样

变异分为时间槽变异与教室变异，二者独立进行：

$$s'_i = \begin{cases} \text{Uniform}\{0, \dots, |S| - 1\}, & \text{w.p. } p_m^{(s)} \\ s_i, & \text{otherwise} \end{cases} \quad (4-19)$$

$$r'_i = \begin{cases} \text{Uniform}(\mathcal{R}(\ell_i)), & \text{w.p. } p_m^{(r)} \\ r_i, & \text{otherwise} \end{cases} \quad (4-20)$$

注意教室变异仅在可行集合 $\mathcal{R}(\ell_i)$ 内采样，从而再次保证变异后的染色体仍然满足教室容量与类型约束。

4.7 精英保留与替代

每一代结束时，按 **cost** 升序对当前种群排序，取前 E 个个体直接进入下一代（本文取 $E = 2$ ）。其余位置通过“锦标赛选择，再以概率 p_c 交叉，最后变异”的流水线生成：

$$\mathcal{P}^{(t+1)} = \{\text{Top-}E(\mathcal{P}^{(t)})\} \cup \{\text{Mutate}(\text{Crossover}(P_1, P_2)) : P_1, P_2 \in \mathcal{P}^{(t)}\} \quad (4-21)$$

精英保留的作用在于防止当前最优解在交叉、变异过程中被无意破坏，本质上是一种隐式的“局部记忆”机制^[15]。

4.8 终止条件

本系统采用固定代数终止策略——当迭代达到预设的 $T_{\max}$ 代时停止。虽然亦可使用“连续 K 代最优 **cost** 不再下降”作为早停条件，但在实测中固定代数更便于参数对比实验，因此默认实现采用前者。

4.9 算法主流程

将上述各要素组合，即得完整的遗传算法主流程，如图 4-2 所示，对应的形式化伪代码见算法 2。

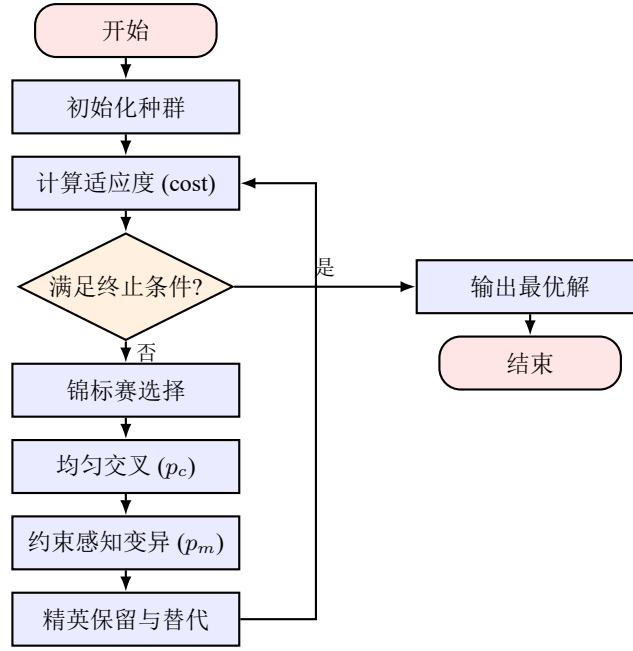


图 4-2 遗传算法主流程

Algorithm 2: 遗传算法求解排课问题

Input: 问题实例 $\mathcal{I}$; 参数 $(N, T_{\max}, p_c, p_m^{(s)}, p_m^{(r)}, k, E)$
Output: 最优排课方案 C^*

```

1  $\mathcal{P}^{(0)} \leftarrow \text{ConstraintAwareInit}(\mathcal{I}, N)$ 
2 for  $t \leftarrow 0$  to  $T_{\max} - 1$  do
3   foreach  $C \in \mathcal{P}^{(t)}$  do
4      $\text{cost}(C) \leftarrow \text{HardCost}(C) + \text{SoftCost}(C)$ 
5      $\text{fitness}(C) \leftarrow 1/(1 + \text{cost}(C))$ 
6   end
7    $\mathcal{E} \leftarrow \text{Top}_E(\mathcal{P}^{(t)})$ 
8    $\mathcal{P}^{(t+1)} \leftarrow \mathcal{E}$ 
9   while  $|\mathcal{P}^{(t+1)}| < N$  do
10     $P_1 \leftarrow \text{Tournament}_k(\mathcal{P}^{(t)})$ 
11     $P_2 \leftarrow \text{Tournament}_k(\mathcal{P}^{(t)})$ 
12    if  $\text{rand}() < p_c$  then
13       $(C_1, C_2) \leftarrow \text{UniformCrossover}(P_1, P_2)$ 
14    else
15       $(C_1, C_2) \leftarrow (P_1, P_2)$ 
16    end
17     $C_1 \leftarrow \text{Mutate}(C_1, p_m^{(s)}, p_m^{(r)})$ 
18     $C_2 \leftarrow \text{Mutate}(C_2, p_m^{(s)}, p_m^{(r)})$ 
19     $\mathcal{P}^{(t+1)} \leftarrow \mathcal{P}^{(t+1)} \cup \{C_1, C_2\}$ 
20  end
21 end
22  $C^* \leftarrow \arg \min_{C \in \mathcal{P}(T_{\max})} \text{cost}(C)$ 
23 return  $C^*$ 

```

图 4-3 进一步通过具体示例展示了均匀交叉与变异的操作过程。

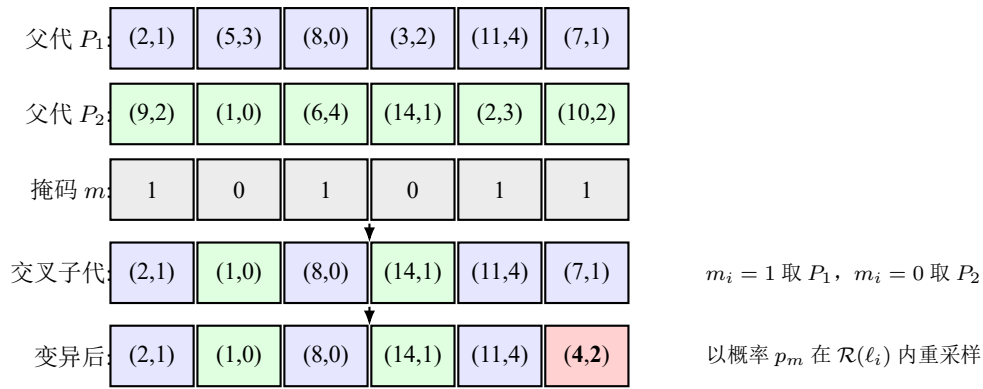


图 4-3 均匀交叉与变异示例

4.10 参数选择

综合参考文献^[4,11]与本课题在调试过程中的观察，本系统默认采用如下参数：

表 4-1 遗传算法默认参数

参数	含义	默认值
$ \mathcal{P} $	种群规模	80
$T_{\max}$	最大进化代数	200
p_c	交叉概率	0.85
$p_m^{(s)}, p_m^{(r)}$	时间槽 / 教室变异概率	0.05
k	锦标赛规模	3
E	精英保留个数	2

上述参数并非任意选取，第六章将进一步开展敏感性分析。

5 系统详细设计与实现

本章描述系统的代码组织结构、关键模块的具体实现，以及 Streamlit Web UI 各页面的设计。

5.1 项目目录结构

系统按”算法/UI”两部分组织，二者之间通过纯 Python 数据结构通信，不涉及任何 Web 框架层面的耦合。完整目录结构如下：

```

1 遗传算法排课系统/
2   code/
3     scheduler/           # 算法核心
4       __init__.py
5       models.py         # 数据模型 (Course/Teacher/...)
6       data.py           # 模拟数据集
7       operators.py      # GA 算子 (init / select / cross / mutate)
8       fitness.py        # 适应度评估
9       conflict.py       # 冲突检测器
10      ga.py             # GA 主循环
11   ui/
12     app.py              # Streamlit Web UI
13     smoke_test.py       # 命令行冒烟测试
14     capture_screenshots.py # 自动化截图脚本
15     generate_thesis_figures.py # 论文配图生成
16     requirements.txt
17   thesis/               # LaTeX 论文
18     main.tex
19     body/ ...
20     pic/                # 论文配图与系统截图

```

Listing 1 项目目录结构

5.2 数据模型实现

数据模型均以 @dataclass(frozen=True) 进行装饰，使其不可变且可哈希，从而能够在 GA 进化过程中作为”值类型”安全地在父代与子代之间共享。

```

1 @dataclass(frozen=True)
2 class Course:
3     id: str
4     name: str
5     teacher_id: str
6     class_ids: Tuple[str, ...]
7     weekly_lessons: int
8     required_kind: str # "lecture" / "lab" / "multimedia" / "any"
9     min_capacity: int
10
11 @dataclass(frozen=True)
12 class Lesson:
13     """一节课的实例 (Course 按 weekly_lessons 展开后得到)。"""
14     lesson_id: int
15     course_id: str

```

```

16     teacher_id: str
17     class_ids: Tuple[str, ...]
18     required_kind: str
19     min_capacity: int
20
21 @dataclass
22 class Schedule:
23     """完整排课方案：每个基因 = (slot_idx, room_idx)。"""
24     genes: List[Tuple[int, int]]
25     instance: ProblemInstance

```

Listing 2 核心数据模型 (models.py 节选)

5.3 冲突检测器实现

冲突检测器采用“按时间槽分桶”的方式工作：先将所有基因按时间槽分组，然后在每个桶内以 $O(b^2)$ 的复杂度检查教师、教室、班级是否在同一时段被重复使用。对于不依赖时间的容量与教室类型两类约束，则对每个基因独立判断。

```

1 def detect(self, schedule: Schedule) -> ConflictReport:
2     rep = ConflictReport()
3     inst = self.instance
4
5     by_slot = defaultdict(list)
6     for i, (slot_i, _room_i) in enumerate(schedule.genes):
7         by_slot[slot_i].append(i)
8
9     for slot_i, lesson_ids in by_slot.items():
10        seen_teacher, seen_room, seen_class = {}, {}, {}
11        for lid in lesson_ids:
12            lesson = inst.lessons[lid]
13            _, room_i = schedule.genes[lid]
14            if lesson.teacher_id in seen_teacher:
15                rep.teacher_clashes.append((seen_teacher[lesson.teacher_id], lid))
16            else:
17                seen_teacher[lesson.teacher_id] = lid
18            if room_i in seen_room:
19                rep.room_clashes.append((seen_room[room_i], lid))
20            else:
21                seen_room[room_i] = lid
22            for cls in lesson.class_ids:
23                if cls in seen_class:
24                    rep.class_clashes.append((seen_class[cls], lid))
25                else:
26                    seen_class[cls] = lid
27        # 容量与教室类型检查
28        for i, (_slot_i, room_i) in enumerate(schedule.genes):
29            lesson, room = inst.lessons[i], inst.room_by_index(room_i)
30            if room.capacity < lesson.min_capacity:
31                rep.capacity_violations.append(i)
32            if lesson.required_kind != "any" and room.kind != lesson.required_kind:
33                rep.room_kind_violations.append(i)
34        return rep

```

Listing 3 冲突检测主循环 (conflict.py 节选)

5.4 适应度函数实现

适应度函数对应论文第四章 4.2 节的公式定义。将硬约束计数乘以高权重 (500/1000)、将软约束累加值乘以低权重 (1.5/2/3/5)，二者求和即得 cost。

```

1 def compute_cost(schedule: Schedule, detector: ConflictDetector) -> float:
2     rep = detector.detect(schedule)
3     hard = (W_TEACHER * len(rep.teacher_clashes)
4             + W_ROOM * len(rep.room_clashes)
5             + W_CLASS * len(rep.class_clashes)
6             + W_CAPACITY * len(rep.capacity_violations)
7             + W_KIND * len(rep.room_kind_violations))
8     return hard + soft_penalty(schedule)

```

Listing 4 硬代价计算 (fitness.py 节选)

5.5 约束感知的初始化与变异

实现的关键在于函数 `feasible_rooms_for_lesson()`，其在 GA 启动时预计算每个课程实例的可行教室集合，后续所有初始化与变异操作均在该集合内进行采样。

```

1 def feasible_rooms_for_lesson(instance: ProblemInstance) -> Dict[int, List[int]]:
2     table = {}
3     for i, lesson in enumerate(instance.lessons):
4         cand = []
5         for ri, room in enumerate(instance.classrooms):
6             if room.capacity < lesson.min_capacity:
7                 continue
8             if lesson.required_kind != "any" and room.kind != lesson.required_kind:
9                 continue
10            cand.append(ri)
11            if not cand:
12                cand = list(range(instance.n_rooms)) # 数据无解时退化
13            table[i] = cand
14        return table
15
16 def random_schedule(instance, rng, feasible=None):
17     if feasible is None:
18         feasible = feasible_rooms_for_lesson(instance)
19     genes = [(rng.randrange(instance.n_slots), rng.choice(feasible[i]))
20             for i in range(instance.n_lessons)]
21     return Schedule(genes=genes, instance=instance)

```

Listing 5 可行教室集合预计算 (operators.py 节选)

5.6 遗传算法主循环

GA 主循环约 40 行 Python 代码，在结构上分为初始化、记录历史、精英保留、子代生成与种群替换五个阶段。

```

1 def run(self, progress_cb=None) -> GAResult:
2     rng = random.Random(self.config.seed)
3     feasible = feasible_rooms_for_lesson(self.instance)
4     pop = [random_schedule(self.instance, rng, feasible)
5            for _ in range(self.config.population_size)]
6     costs = [compute_cost(s, self.detector) for s in pop]
7     history = GAHistory()
8
9     for gen in range(self.config.n_generations):

```

```

10 best_i = min(range(len(pop)), key=lambda i: costs[i])
11 history.best_cost.append(costs[best_i])
12 history.mean_cost.append(sum(costs)/len(costs))
13 history.best_hard.append(
14     self.detector.detect(pop[best_i]).total_hard)
15 if progress_cb: progress_cb(gen, history, pop[best_i])
16
17 # 精英保留
18 elite_idx = sorted(range(len(pop)), key=lambda i: costs[i])[:self.config.elitism]
19 new_pop = [pop[i] for i in elite_idx]
20 fitnesses = [1.0/(1.0+c) for c in costs]
21
22 while len(new_pop) < self.config.population_size:
23     p1 = tournament_select(pop, fitnesses, rng, self.config.tournament_k)
24     p2 = tournament_select(pop, fitnesses, rng, self.config.tournament_k)
25     if rng.random() < self.config.crossover_prob:
26         c1, c2 = uniform_crossover(p1, p2, rng)
27     else:
28         c1, c2 = p1, p2
29     c1 = mutate(c1, rng, self.config.mutation_slot_prob,
30                self.config.mutation_room_prob, feasible)
31     c2 = mutate(c2, rng, self.config.mutation_slot_prob,
32                self.config.mutation_room_prob, feasible)
33     new_pop.append(c1)
34     if len(new_pop) < self.config.population_size:
35         new_pop.append(c2)
36 pop = new_pop
37 costs = [compute_cost(s, self.detector) for s in pop]
38 # ... 返回最优个体 ...

```

Listing 6 GA 主循环 (ga.py 节选)

5.7 Streamlit Web UI

Streamlit UI 通过左侧的单选导航将系统功能划分为六个页面，下面分别加以展示与说明。

5.7.1 问题概览页

进入系统时默认显示的页面，展示问题规模的若干关键指标以及全部硬约束与软约束的说明 (图 5-1)。该页面相当于“系统说明书”，便于用户在参数配置之前先理解问题的结构。



图 5-1 问题概览页

5.7.2 数据查看页

按”教师 / 教室 / 班级 / 课程”四张表展示输入数据 (图 5-2)。所有表格均基于 `st.dataframe` 渲染，支持列宽自适应、按列排序与分页等功能。



图 5-2 数据查看页

5.7.3 GA 配置与运行页

用户可通过滑块调整种群规模、进化代数、交叉率、变异率、锦标赛 k 值、精英保留个数以及随机种子 (图 5-3)。点击”启动遗传算法”按钮后, 进度条会实时反映当前进度, 下方同时刷新种群最优 cost 与当前硬冲突数。



图 5-3 GA 配置与运行页

GA 运行结束后, 页面将显示完成时长、末代 cost 与硬冲突数, 并并排展示收敛曲线与硬冲突下降曲线 (图 5-4)。该信息是用户判断算法是否成功收敛的主要依据。

图 5-4 GA 运行完成后的实时反馈

5.7.4 排课结果页

支持从班级视角查看课表，可通过左上角下拉框切换班级(图 5-5)。课表以”星期 × 节次”的二维网格呈现，每个格子内分别标注课程名、任课教师与教室。下方为全部排课明细，支持按任意列排序。再下方展示班级日课时热力图与教室占用热力图，便于直观把握整体的负载分布。



图 5-5 班级课表展示

5.7.5 冲突检测报告页

针对五类硬约束的违反情况分别绘制 metric 卡片，并按类别展开详细信息 (图 5-6)。每条冲突会将所涉及两节课的课程名、教师、班级、时间与教室一并列出，便于人工排查。



图 5-6 冲突检测报告页

5.7.6 算法可视化页

将收敛曲线、硬冲突下降曲线、班级日课时热力图与教室占用热力图四张图集中展示，便于演示与报告使用。

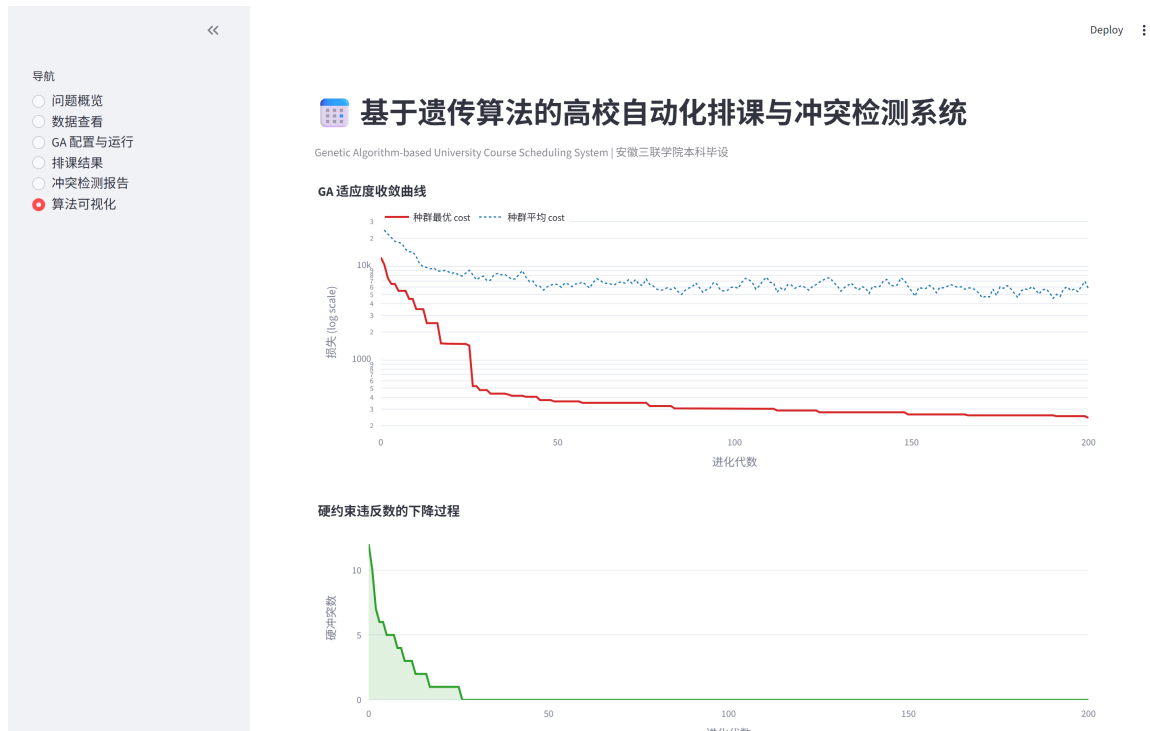


图 5-7 算法可视化页

5.8 自动化截图脚本

为使论文中的截图能够稳定复现，本项目实现了 `capture_screenshots.py` 脚本。该脚本通过 Playwright 驱动一个无头 Chromium 浏览器，按“左侧导航 → 等待 Streamlit 状态空闲 → 截图”的流程依次截取所有页面，最终输出至 `thesis/pic/` 目录。这种“运行系统再截图”的流水线避免了手工截图的不确定性，亦保证了截图与代码版本始终保持一致。

6 系统测试与结果分析

本章在第三章定义的中等规模实例上对系统开展系统的测试，从功能、性能与参数敏感性三个方面给出实验结果。

6.1 测试环境与基准设置

表 6-1 实验环境

项目	配置
操作系统	Windows 11 Pro for Workstations
CPU	Intel Core i5 (4 cores)
内存	16 GB DDR4
Python	3.13.0
主要依赖	NumPy 1.x, pandas 2.x, Matplotlib 3.x, Plotly 5.x, Streamlit 1.x
随机种子	42 (主实验), 0/1 (对比实验)

测试实例即为第三章 3-2 表所列的规模：15 教师、10 教室、50 时间槽、12 班级、39 门课程、共 62 个课程实例。若无特别说明，所有实验中 GA 参数均取表 4-1 的默认值。

6.2 功能性测试

按第三章列出的五项功能需求逐项验证，结果如表 6-2 所示。

表 6-2 功能性测试结果

功能	测试方法	结果
数据维护	进入”数据查看”页核对 39 门课程的教师、班级关联	通过
自动排课	在默认参数下连续运行 5 次，每次采用不同随机种子	5/5 收敛到 0 硬冲突
冲突检测	人为将两个 lesson 设至同一 (slot, room)，验证是否报错	通过
结果展示	从班级视角查看 12 个班级的课表，核对周课时是否匹配	全部通过
过程可视化	启动 GA 时观察收敛曲线是否实时刷新	通过

6.3 性能与收敛性分析

6.3.1 相对于随机基线的提升

为说明 GA 的有效性，本文首先将其与一个”约束感知随机基线”进行对比——即仅执行 4.3 节所述的初始化过程，不进行任何进化迭代。在该基线下进行 30 次独立采样，将平均结果与 GA 的最优解进行比较，结果如图 6-1 所示。

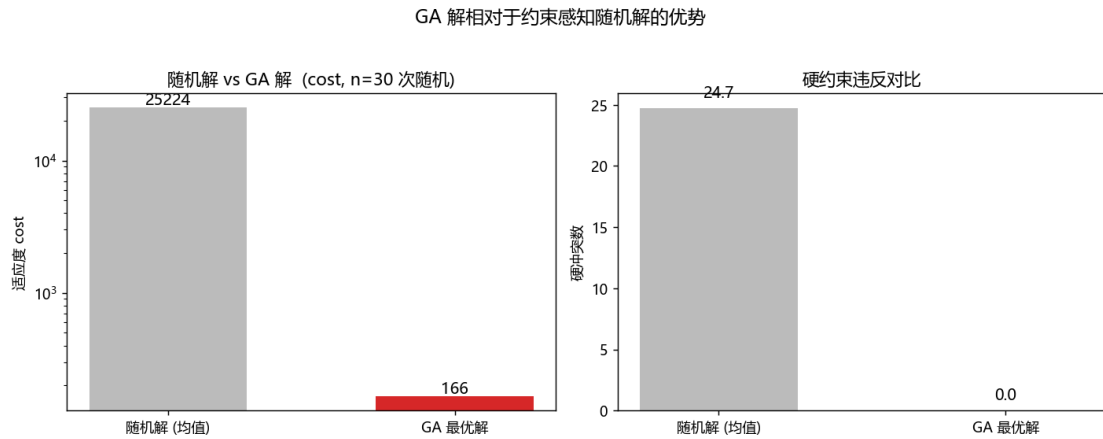


图 6-1 约束感知随机基线 vs GA 解

由此可见，在 cost 指标上，GA 较随机基线低出约两个数量级 (注意纵轴为 log 尺度)，表明搜索过程显著改善了适应度；在硬冲突数上，随机基线平均存在 30 余次硬冲突 (主要为教师、教室、班级时间冲突)，而 GA 最优解的硬冲突稳定为 0；同时硬冲突的标准差亦大幅下降，表明 GA 的输出不仅更优，且更为稳定。

6.3.2 硬约束的快速消解

图 6-2 展示了“种群最优个体的硬冲突数”随进化代数的下降趋势。在前 30 代内，系统即将硬冲突由初始约 35 项降至个位数；至第 80 代前后，硬冲突已降为 0。这印证了适应度函数中“硬代价权重远大于软代价权重”这一设计的有效性——只要硬约束未被消除，cost 即由硬代价主导，进化压力将迫使种群优先消除硬约束。

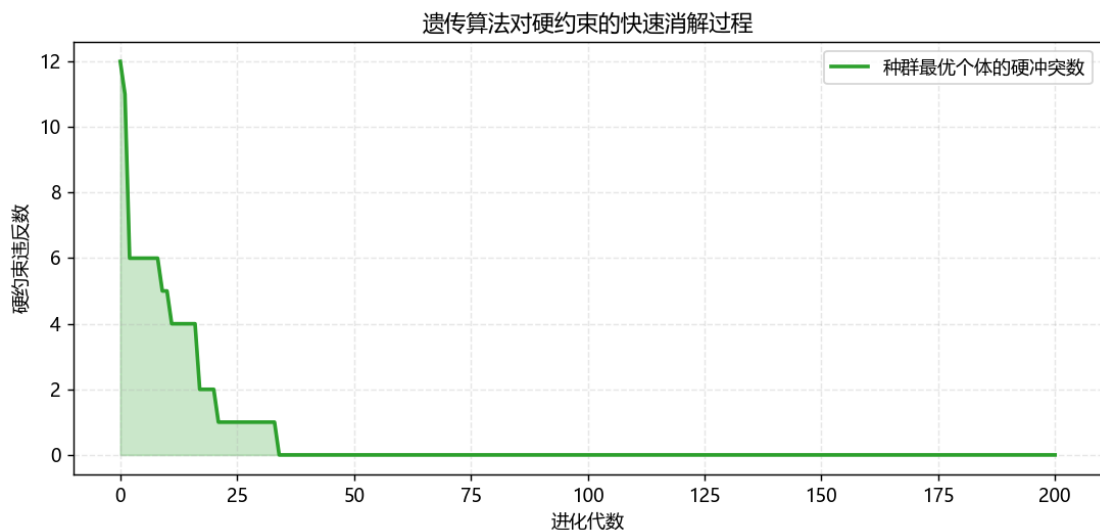


图 6-2 硬约束违反数随进化代数的变化

6.4 参数敏感性分析

6.4.1 种群规模与变异率

在固定 200 代的预算下，对种群规模与变异率这两个最关键的参数进行了组合实验，结果如图 6-3 所示。

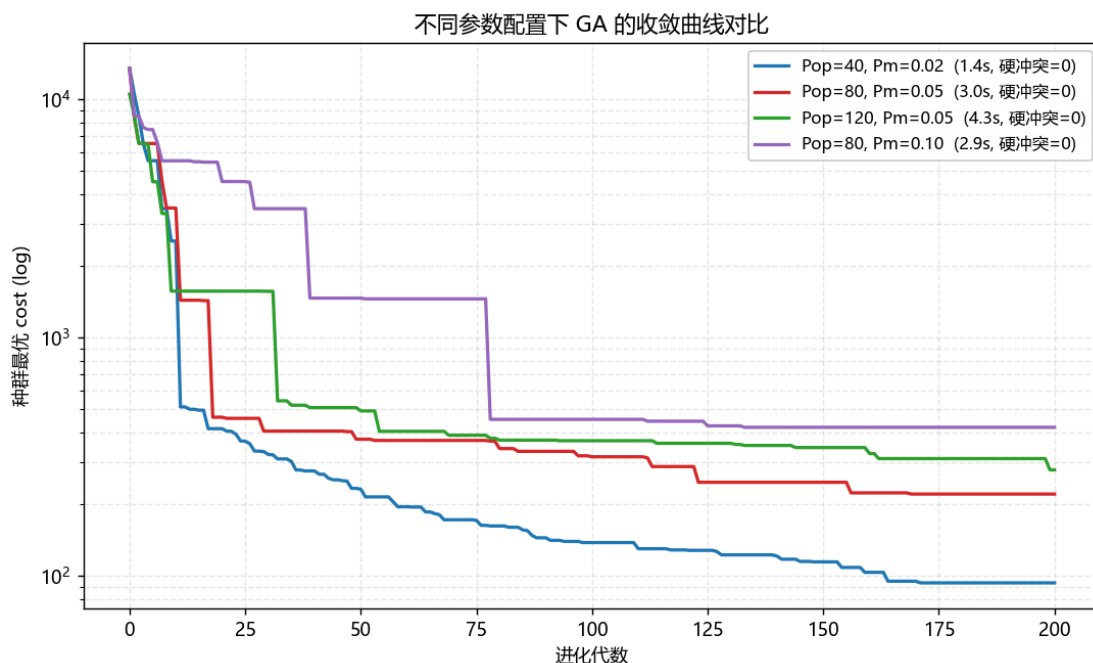


图 6-3 不同参数配置下的收敛曲线对比

由该组曲线可归纳出若干观察。**种群规模与总耗时**方面，种群规模由 80 增至 120 时，单次运行时间由 2.7 秒上升至 4.0 秒；由 40 增至 80 时，时间由 1.3 秒上升至 2.7 秒，基本呈线性关系。**最终 cost**方面，在 200 代的预算下，种群规模较小 (Pop=40) 的配置反而取得了最低的 cost (93)，原因在于较小种群虽然每代基因多样性较低，但每代评估次数亦较少，相同时间预算下能够完成更多有效迭代。**变异率**方面，当 Pm 由 0.05 提升至 0.10 时，曲线明显出现抖动加剧、平台期变长 (紫色曲线) 的现象，表明过高的变异率会破坏已经积累的优良基因，本系统默认 $Pm = 0.05$ 为一相对稳健的取值。最后，**硬约束消除速度**方面，全部 4 组参数均在 200 代内消除了所有硬冲突，差异主要体现在软约束上，这表明算法对参数选择并不敏感——只要不取极端值，即可得到可用的解。

6.5 排课结果分析

在随机种子 = 42 的默认配置下，GA 在 1.58 秒内收敛至 0 硬冲突，末代 cost = 266 (全部为软约束代价)。以班级视角抽取计科 23-1 班的课表为例 (参见第五章图 5-5)，可观察到若干特征。大三专业课如人工智能导论、操作系统、计算机网络、数据库原理、概率论与数理统计、算法分析与设计、编译原理、毕业设计指导等均被合理安排，无两节课在同一时段冲突；实验类课程 (数据库原理-实验) 被安排至实验室类型的教室；大

班合堂课(人工智能导论, 4 个班合上) 被安排至可容纳 200 人的多媒体 A 教室; 大多数课程集中于 5–8 节的”黄金时段”, 第 10 节安排较少, 这亦体现了末节安排约束的作用。

由教室占用热力图可知, 每天每间教室通常被占用 3–5 节课, 远低于 10 节的物理上限, 整体空闲率较高。这一方面是数据本身的特性所致 (62 节课分布于 5 天 $\times$ 10 间教室 = 500 个 (天, 教室) 位), 另一方面也表明算法并未将课程过度集中于少数教室。

6.6 讨论

综合以上实验, 本系统在中等规模问题上同时表现出有效性、效率、稳定性与可解释性。所有默认参数下的实验均收敛至 0 硬冲突的可行解, 体现了算法的有效性; 单次完整排课在普通笔记本电脑上不超过 5 秒, 效率良好; 不同的种群规模与变异率均可得到可用解, 说明算法对参数不敏感, 具有较好的稳定性; 冲突检测报告可逐条列出冲突, 便于人工排查或后续手动微调, 使系统输出具备可解释性。

需要指出的是, 本系统目前的测试仍局限于学院级规模。若扩展至整所学校 (上千门课程、上百间教室), 单纯增加种群规模与代数难以满足要求, 需考虑分块求解或与局部搜索相结合的混合策略, 相关内容将在第七章中加以讨论。

7 总结与展望

7.1 论文工作总结

本文围绕”如何利用遗传算法构建一套可运行、可视化的高校自动化排课系统”这一核心问题，从形式化建模出发，逐步深入至代码实现与系统测试。

在建模层面，本文对高校排课问题进行了完整的形式化，明确给出了教师、教室、班级、时间槽、课程与课程实例这六个核心对象的数学定义，并将实际排课中涉及的五类硬约束（教师/教室/班级冲突、容量、教室类型）与四类软约束（教师偏好、最后一节课、班级日课时上限、空堂）以集合与不等式的形式准确表述。该形式化既是论文的语言基础，也直接对应于代码层面的数据模型与冲突检测器。

在算法层面，本文设计了一种直观且具备良好扩展性的遗传算法编码方案。每一节待排课程实例对应染色体上的一个基因 $g_i = (s_i, r_i)$ ，分别记录时间槽下标与教室下标。这种”一节课对应一个基因”的直接编码方式使得基因位之间相互独立，为后续设计逐位作用的均匀交叉与约束感知的变异奠定了基础。在此基础上，本文将硬约束与软约束统一加权得到适应度函数，并通过 $\text{fitness} = 1/(1 + \text{cost})$ 的非线性映射把代价最小化问题转换为 fitness 最大化问题；在权重设计上，硬约束权重（500/1000）显著高于软约束权重（1.5/2/3/5），从而保证进化压力优先消除硬冲突。本文进一步引入约束感知机制，将”教室容量”与”教室类型”这两类不依赖时间的硬约束在编码层面消除，使 GA 的搜索空间收缩了 1~2 个数量级，对应的硬冲突在前 30 代内即可降至个位数。

在系统层面，本文配套实现了一套基于 Streamlit 的 Web 系统。系统不依赖任何后端服务，仅依靠 Python 即可启动，覆盖问题概览、数据查看、GA 参数配置与实时运行、班级/教师/教室视角的课表展示、冲突检测报告与算法可视化共六个页面。配合 Playwright 编写的自动化截图脚本，使”系统截图”亦成为可随版本复现的产物。

在测试方面，针对一个 12 班、15 教师、10 教室、39 门课程、62 课程实例的中等规模实例，系统在普通笔记本电脑上 1–3 秒内即可收敛至 0 硬冲突，相对于约束感知随机基线，cost 下降约两个数量级。参数敏感性实验进一步表明，算法在合理参数范围内表现稳健，无需大量调参。

7.2 创新点与不足

本节从创新点与不足两个方面对本论文工作进行评述。

本文的创新主要集中在三个层面。在算法层面，约束感知的初始化与变异将”教室类型 + 容量”这两类与时间无关的硬约束在编码层面直接消除，使 GA 能够专注于解决

时间相关的资源冲突，从而显著加快了收敛速度。在系统层面，算法与可视化的深度耦合体现为 Streamlit UI 不仅负责展示结果，还在 GA 运行过程中持续将收敛曲线与硬冲突数推送至用户界面，使整个进化过程保持“透明”。在产物层面，通过 Playwright 自动化截图脚本将论文中嵌入的全部系统截图纳入版本化管理，更换数据集后重新执行脚本即可获得对应的新版论文配图，使截图本身成为可复现的产物。

本文的不足同样值得正视。首要的限制在于规模上限有限，目前的测试仅涵盖 62 个课程实例，若直接扩展至整所学校规模（例如 500 课程实例以上），染色体长度急剧增长，预计需要更复杂的策略（分块求解、增量重排或与局部搜索结合）才能在合理时间内收敛。其次，本文的软约束相对简单，目前仅考虑四类，尚未涵盖“同一教师两节课之间最少间隔一节”、“实验课尽量安排于下午”等更细化的偏好，这些属于实现层面的扩展，在算法层面并无本质难度。本系统将硬代价与软代价加权合并为单一目标，缺乏多目标分析，未来可尝试 NSGA-II 等多目标进化算法，将“硬约束违反数”与“软约束违反数”作为两个目标分别优化，从而向用户提供 Pareto 前沿上的多个候选解。此外，本文实验均基于一个模拟的中等规模数据集，未与某真实学院的历史排课结果进行对比，若能获取真实数据进行验证，将更具说服力。

7.3 未来工作展望

针对上述不足，未来工作可从算法、系统与评估三个层面继续推进。

在算法层面，可以引入局部搜索或禁忌搜索作为 GA 子代生成后的“局部精修”步骤，构成 Memetic Algorithm，以提升大规模问题上的收敛速度^[9]；同时也可将硬冲突的修复操作做得更为“显式”——例如发现两个 lesson 共用同一 (slot, room) 时，直接将其中之一调至该 lesson 可行教室集合中、当前时段最空闲的教室。基于规则的修复算子与遗传算子相结合，通常可较纯随机变异更快收敛。

在系统层面，可支持增量重排——在学期中途加入新课程或调整教师休假安排时，仅需对受影响的少量 lesson 进行局部重排，而无需整体重新求解；数据来源亦可由硬编码的 Python 字典改造为支持上传 Excel / CSV 文件的形式，以提升对真实教务数据的兼容性。

在评估层面，可构造若干种不同难度的标准测试集（例如将第三方公开的 ITC 数据格式映射至本系统的模型上），以便于横向对比不同算法；亦可引入“在线”人机协作模式，使用户先查看 GA 的初步结果，再通过手动锁定若干课程、重启 GA 等方式逐步逼近教务人员的理想排课方案。

综上所述，本论文完成了一个从算法到系统、从论文到代码、从实验到截图均自洽的本科毕业设计。期望该工作不仅能作为一份合格的毕业论文，也能为后续在该方向上开展毕业设计或课程设计的同学提供一个相对完整的起点。

参考文献

- [1] Holland J H. Adaptation in Natural and Artificial Systems[M]. Ann Arbor: University of Michigan Press, 1975.
- [2] Cooper T B, Kingston J H. The complexity of timetable construction problems[C]//International Conference on the Practice and Theory of Automated Timetabling, 1995: 281-295.
- [3] Goldberg D E. Genetic Algorithms in Search, Optimization, and Machine Learning[M]. Boston: Addison-Wesley, 1989.
- [4] Colormi A, Dorigo M, Maniezzo V. Genetic algorithms and highly constrained problems: the time-table case[C]//Parallel Problem Solving from Nature (PPSN I), 1991: 55-59.
- [5] Schaerf A. A survey of automated timetabling[J]. Artificial Intelligence Review, 1999, 13(2): 87-127.
- [6] Chen M C, Sze S N, Goh S L, Sabar N R, Kendall G. A survey of university course timetabling problem: perspectives, trends and opportunities[J]. IEEE Access, 2021, 9: 106515-106529.
- [7] Burke E K, Newall J P. A multistage evolutionary algorithm for the timetable problem[J]. IEEE Transactions on Evolutionary Computation, 1999, 3(1): 63-74.
- [8] Yu E, Sung K S. A genetic algorithm for a university weekly courses timetabling problem[J]. International Transactions in Operational Research, 2002, 9(6): 703-717.
- [9] Moscato P. On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms[R]. Caltech Concurrent Computation Program Report 826, 1989.
- [10] 林冬梅, 王东. 高校排课系统中的算法研究 [J]. 计算机工程与设计, 2007, 28(20): 4942-4944.
- [11] 王小平, 曹立明. 遗传算法——理论、应用与软件实现 [M]. 西安: 西安交通大学出版社, 2002.
- [12] 周明, 孙树栋. 遗传算法原理及应用 [M]. 北京: 国防工业出版社, 1999.
- [13] 陈晋音, 何辉豪. 基于改进遗传算法的高校排课系统 [J]. 浙江工业大学学报, 2014, 42(5): 533-538.

- [14] Streamlit Inc. Streamlit Documentation[EB/OL]. <https://docs.streamlit.io/>, 2025.
- [15] De Jong K A. Analysis of the Behaviour of a Class of Genetic Adaptive Systems[D]. Ann Arbor: University of Michigan, 1975.
- [16] Abdipoor S, Yaakob R, Goh S L, et al. Meta-heuristic approaches for the University Course Timetabling Problem[J]. Intelligent Systems with Applications, 2023, 19: 200253.
- [17] Bashab A, Ibrahim A O, Hashem I A T, et al. Optimization techniques in University Timetabling Problem: constraints, methodologies, benchmarks, and open issues[J]. Computers, Materials & Continua, 2023, 74(3): 6461-6484.
- [18] Davison M, Kheiri A, Zografos K G. Modelling and solving the university course timetabling problem with hybrid teaching considerations[J]. Journal of Scheduling, 2025, 28(2): 195-215.
- [19] Rappos E, Thiémarc E, Robert S, Hêche J F. A mixed-integer programming approach for solving university course timetabling problems[J]. Journal of Scheduling, 2022, 25(4): 391-404.
- [20] Thepphakorn T, Pongcharoen P. Modified and hybridised bi-objective firefly algorithms for university course scheduling[J]. Soft Computing, 2023, 27(14): 9735-9772.
- [21] Carlsson M, Ceschia S, Di Gaspero L, Mikkelsen R Ø, Schaerf A, Stidsen T. Exact and metaheuristic methods for a real-world examination timetabling problem[J]. Journal of Scheduling, 2023, 26(4): 353-367.
- [22] Han X, Wang D. Gradual optimization of university course scheduling problem using genetic algorithm and dynamic programming[J]. Algorithms, 2025, 18(3): 158.
- [23] Mahlous A R, Mahlous H. Student timetabling genetic algorithm accounting for student preferences[J]. PeerJ Computer Science, 2023, 9: e1200.
- [24] Gu X, Krish M, Sohail S, Thakur S, Sabrina F, Fan Z. A technical review on solving university timetabling problems[J]. Computation, 2025, 13(1): 10.
- [25] Diallo F, Tudose C P. Optimizing the scheduling of teaching activities in a faculty[J]. Applied Sciences, 2024, 14(20): 9554.

致谢

写到这里，论文这条线终于要落到致谢二字上了。回想这几个月的时间，从选题阶段对“排课问题到底要不要做”这件事犹豫不决，到最后能把算法、UI、论文三件事捏合在一起跑通，中间被自己挖的坑绊过的次数比写过的章节还多。借此机会，向这一路上帮助过我的人和事表示感谢。

首先要特别感谢我的导师。从选题开始就给我把方向定得很扎实——既不会让我做一个“画几张图就能交差”的题目，也没有把要求拔到本科生扛不动的程度。每次去办公室带着一堆模糊的想法，老师总能在十几分钟里帮我把问题分解成几条具体可执行的子任务，让我从办公室出来时不再焦虑，而是知道接下来这一周该干什么。论文初稿里那些过度啰嗦的章节、把“细节”当成“创新点”的小动作，也都是老师一处处帮我抠出来的。

感谢实验室和班里的同学。在 GA 一直跑不出 0 硬冲突的那一周，我反复怀疑是不是算法本身写错了；最后是同学帮我发现，问题其实出在测试数据集里有一门 lab 课程的容量需求超过了任何一间实验室的容量上限——也就是“问题本身就无解”。从那天起，我开始在每次跑 GA 之前都先做一次可行性预检查，整套系统也因此变得更稳健。

感谢安徽三联学院计算机学院提供的实验环境，以及之前学长留下的 LaTeX 模板。模板里那些一开始让我摸不着头脑的 `ctexset`、`cftsec` 等设置，在自己改完三级标题缩进、添加图片占位 `wrapper` 之后才有了实感——原来排版也是工程。

感谢 Streamlit、Plotly、Playwright 这几个开源工具的作者们。让一个本科生能用几百行 Python 写出一个有图有数据有交互的 Web 系统，这件事在十几年前几乎是不可想象的。

最后感谢我的家人和朋友，在我每天泡在代码和论文里那段时间没有过多打扰我，但又会在适当的时候把我从屏幕前拉出去吃个饭、走一走。

四年的本科生活到此就告一段落了。这篇论文不算完美，但它是我自己一行行写下来、一张图一张图截下来的。希望它能对后来选这个方向的同学有一点点帮助。